Self-Supervised Adaptation Method to Concept Drift for Network Intrusion Detection

Shuo Yang, Xinran Zheng, Jinze Li, Jinfeng Xu, Xinchen Zhang, and Edith C. H. Ngai, *Senior Member, IEEE*

***Abstract*—The deployment of learning-based models to detect malicious activities in network traffic flows is significantly challenged by concept drift. With evolving attack technology and dynamic attack behaviors, the underlying data distribution of recently arrived traffic flows deviates from historical empirical distributions over time. Existing approaches depend on a significant amount of labeled drifting samples to facilitate the deep model to handle concept drift, which faces labor-intensive manual labeling and the risk of label noise. In this paper, we propose ReCDA, a Concept Drift Adaptation method with Representation enhancement, which consists of a self-supervised representation enhancement stage and a weakly-supervised classifier tuning stage. Specifically, in the initial stage, ReCDA introduces drift-aware perturbation and representation alignment to facilitate the model in acquiring robust representations from drift-aware and drift-invariant perspectives. In the subsequent stage, a meticulously crafted instructive sampling strategy and a robust representation constraint encourage the model to learn discriminative knowledge about benign and malicious activities during fine-tuning, thereby enhancing performance further. We conduct comprehensive offline and online evaluations on several benchmark datasets under varying degrees of concept drift. The experiment results demonstrate the superior adaptability and robustness of the proposed method.**

***Index Terms*—Intrusion detection, network security, concept drift.**

I. Introduction

NETWORK Intrusion Detection (NID) plays an essential role in ensuring security by continuously identifying and discarding suspicious activities in network traffic [2], [3]. Deep Learning (DL) has emerged as a promising approach for intrusion detection due to its capability to learn complex representations from raw traffic flows [4], [5], [6]. In recent years, DL-based intrusion detection systems have demonstrated state-of-the-art performance [7], [8] and have found wide applications in diverse domains [9], [10], [11], [12].

Received 13 September 2024; revised 9 August 2025; accepted 11 August 2025. Date of publication 14 August 2025; date of current version 4 November 2025. This work was supported in part by the Hong Kong UGC General Research Fund under Grant 17203320 and Grant 17209822 and in part by the project grants from the HKU-SCF FinTech Academy. An earlier version of this paper was presented in part at the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2024) [DOI: 10.1145/3637528.3672007]. *(Corresponding author: Edith C. H. Ngai.)*

Shuo Yang, Jinze Li, Jinfeng Xu, Xinchen Zhang, and Edith C. H. Ngai are with the Department of Electrical and Electronic Engineering, The University of Hong Kong, Hong Kong, China (e-mail: shuoyang.ee@gmail.com; chngai@eee.hku.hk).

Xinran Zheng is with Shenzhen International Graduate School and Department of Electronic Engineering, Tsinghua University, Beijing 100190, China.

Digital Object Identifier 10.1109/TDSC.2025.3599321

In today's interconnected digital landscape, the relentless evolution of cyber threats [13] demands robust intrusion detection systems capable of adapting to the ever-changing nature of concept drift. Concept drift refers to the phenomenon in which the statistical properties of monitored data change over time [14], [15], posing significant challenges to intrusion detection systems that rely on static models [16], [17]. Failure to adapt to concept drift may have severe consequences, including increased false positives or missed intrusions [18], [19], potentially resulting in compromised systems and sensitive data breaches [20].

Traditional ensemble techniques [21], [22] for addressing concept drift in intrusion detection have limitations in adaptability and robustness. These approaches often struggle to keep pace with the dynamic and complex nature of modern attacks, since they fail to capture the nuanced patterns and subtle changes that occur over time [23]. To bridge this gap, concept drift adaptation through incremental learning or continuous learning [24], [25], [26], [27] shows promise by introducing new knowledge into the classifier to reduce the cumulative prediction error. Although these approaches can bring some performance improvement, they come at the cost of increased manual labeling. Some existing work [26], [28], [29] used pseudo-labels to update the model, which is more vulnerable to label noise and hence susceptible to drastic performance deterioration by self-poisoning [30]. Consequently, there is an urgent need for novel and effective concept drift adaptation techniques to enhance the robustness and reliability of intrusion detection systems.

In this paper, to address the aforementioned challenges, we proposed ReCDA:[1] a self-supervised concept drift adaptation method with representation enhancement. Different from existing work [17], [26], [28], we leverage the inherent characteristics of collected traffic flows to guide the model in extracting both drift-aware and drift-invariant representations, thus circumventing labor-intensive manual labeling and the risk of noisy labels. Technically, we initially employ perturbation techniques to generate the drift view of the original traffic flows, inspired by the observation that changes in local features contribute to concept drift. As illustrated in Fig. 1, discrepancies between attack variants are often manifested in the feature distribution. Subsequently, the original flow and its perturbed version are fed into a shared encoder network optimized by contrastive loss to align the representations. This allows us to acquire a robust feature extractor capable of narrowing the gap between historical

[1] https://github.com/kalendsyang/ReCDA.git

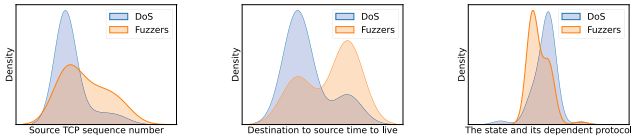


Fig. 1. Feature distribution of DoS and fuzzers attacks.

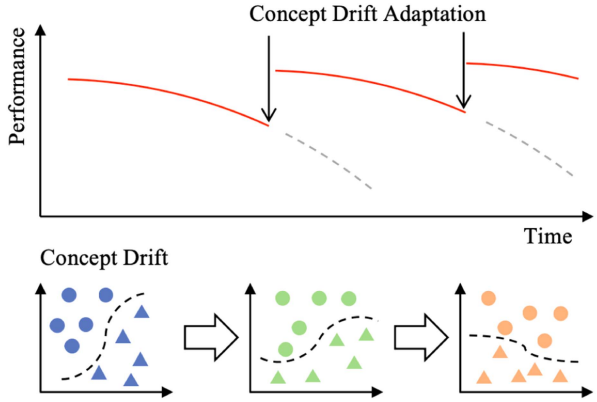


Fig. 2. Concept drift adaptation diagram.

and drift traffic flows while maintaining the separability of instances. To impart discriminative knowledge about benign and malicious activities, we sample some instructive instances from the historical dataset. Given the robustness of the representations obtained in the representation enhancement stage, we utilize consistency regularization to constrain the expansion of the representation space, thus mitigating the risk of overfitting historical distribution. Through the above process, the model can obtain a representation that is both conscious of concept drift and resilient to it, thereby improving the classification performance and robustness of the model in concept drift scenarios.

The contributions of this paper are as follows:

- We propose an advanced concept drift adaptation method with representation enhancement. Our method benefits from the representations with drift-aware and drift-invariant, thereby enhancing the performance of the intrusion detection model.
- We provide a novel perspective on concept drift adaptation through feature perturbation. To the best of our knowledge, it is the first self-supervised concept drift adaptation method designed for network intrusion detection, circumventing labor-intensive manual labeling and the risk of label noise.
- We highlight the mild and insufficient nature of existing evaluations of concept drift adaptation methods, which have only been tested under limited degrees of drift. In contrast, we propose a more rigorous evaluation setting.
- We conduct extensive offline experiments on several intrusion detection datasets under varying degrees of concept drift. The experiment results demonstrate the superior adaptability and robustness of the proposed method.
- We further conduct online experiments on the latest benchmark concept drift dataset. The experiment results demonstrate that ReCDA can significantly alleviate the decline in model performance caused by concept drift.

II. Related Work

A. Network Intrusion Detection

Network intrusion detection is a potent technology for detecting and responding to malicious and unauthorized activities within networks. Recently, numerous learning-based methods [31], [32], [33], [34], [35], [36], [37], [38] have been proposed to safeguard networks against advanced attacks. Qiu et al. [32] observed entangled distributions of flow features and proposed a two-step feature disentanglement approach along with a dynamic graph diffusion scheme to identify various attacks. Mirsky et al. [34] introduced Kitsune, an unsupervised online network intrusion detection method. It employs an ensemble of autoencoders to collectively distinguish between benign and malicious traffic patterns. Fu et al. [35] utilized frequency domain features to develop a real-time machine learning-based malicious traffic detection system that ensures high detection accuracy and throughput.

As the complexity of network environments continues to increase, robust and reliable intrusion detection methods have garnered considerable attention. Wang et al. [39] proposed an adversarial sample detection method based on manifold and decision boundaries to deal with adversarial sample attacks. Houda et al. [40] leverages Explainable Artificial Intelligence (XAI) techniques and Blockchain to secure federated learning-based IDS in the IoT networks. Considering the increase in attack variants and the blurring of the boundary between malicious and benign activity, Yue et al. [37] designed heuristic contrastive tasks to mine semantic relationships among samples. Diallo et al. [38] utilized cluster centers to expand the features of a given dataset, which improved the robustness and the generalization abilities of detection models. Despite the superior performance of these approaches, they rely on the assumption of data stationarity in a static intrusion detection environment [41]. Consequently, they cannot be applied in dynamic environments experiencing concept drift.

B. Concept Drift Adaptation

As shown in Fig. 2, concept drift refers to the changes in the statistical properties of data features over time [14], which can lead to a gradual decline in the performance of models that were well-trained on historical data. To address this challenge, concept drift detection and adaptation techniques are required. Concept drift detection aims to identify when concept drift occurs in the data. The detection methods typically monitor the model's performance [42] or track the statistical properties of the incoming data [17], [43], [44]. When significant deviations from the expected behavior are observed, it indicates the presence of concept drift. For example, CADE [17] uses contrastive learning to map the data samples into a low-dimensional space and learns a distance function to measure dissimilarity between samples, which presented satisfactory detection performance in security

applications. After concept drift is detected, adaptation strategies need to be employed to update the model and mitigate its impact, which is crucial to improving model robustness.

The goal of concept drift adaptation is to make models resilient to evolving data distributions. There are two primary approaches to handling this challenge: incremental learning [24], [45], [46] and ensemble learning [21], [23], [47], [48]. Ensemble learning combines multiple base learners to construct an ensemble model that offers better generalization. However, these methods are still based on detecting known patterns and may fail to capture the nuanced patterns and subtle changes that occur over time. In contrast, incremental learning methods address concept drift by retraining or altering the model to accommodate both drift data and historical data. For example, Chen et al. [24] proposed a continual learning method to combat the concept drift problem of Android malware classifiers, which applies similarity-based uncertainty to select new samples for analysts to label and retrain the classifier. Han et al. [49] performed hypothesis testing on the model's output distribution to detect significant changes in the data distribution and selected influential samples for manual labeling, enabling adaptation to drift. It is important to note that their method is specifically designed to address normality drift (changes in benign samples) and does not consider the evolution of attacks, which limits its practical applicability in real-world intrusion detection scenarios.

Despite their effectiveness, these methods often rely on manual labeling, which can be very expensive in security applications. To alleviate this, Andresini et al. [26] proposed the use of the nearest centroid neighbor strategy to generate pseudo-labels, achieving satisfactory performance in network intrusion detection. However, their approach is highly dependent on accurate clustering of drifted samples and is vulnerable to label noise and self-poisoning of pseudo-labels [30]. Zhao et al. [50] transformed the known/new category identification problem into multiple independent one-class learning tasks, using ensemble clustering for label assignment, facilitating incremental updates to intrusion detection models. However, maintaining multiple one-class classifiers requires significant resources, and as the number of categories increases, the difficulty of expanding also grows. In recent years, self-supervised learning has emerged as a promising alternative for concept drift adaptation, particularly in domains where labeled data is scarce or expensive to obtain. For example, self-supervised concept drift adaptation methods have been successfully applied in object detection [51], [52], time-series modeling [53], and IoT malware detection [44]. However, these methods often involve domain-specific processing and augmentation techniques, which may not be directly suitable for network traffic.

To bridge these gaps, we propose ReCDA, a self-supervised concept drift adaptation method with representation enhancement. A comparison of representative related works is shown in Table I. Unlike existing approaches [17], [26], [50], we leverage the inherent characteristics of network traffic flows to guide the model in extracting both drift-aware and drift-invariant representations. This allows ReCDA to circumvent the need for labor-intensive manual labeling and the risk of noisy labels. ReCDA is highly flexible, functioning either as an independent intrusion detector to reduce the impact of concept drift or as a plug-and-play module within existing continuous learning frameworks to enhance intrusion detection performance. ReCDA provides a robust and efficient solution for adapting to evolving attack strategies without the need for extensive manual labeling.

TABLE I
COMPARISON OF REPRESENTATIVE RELATED WORKS

Features	CADE [17]	INSOMIA [26]	TRIDENT [50]	**ReCDA**
Labeling-free	○	◐	◐	●
Resource-efficient*	●	○	○	●
Plug-and-Play	○	○	○	●

●: true, ◐: partially true, ○: flase; * meatured in Section V-C.

III. METHODOLOGY

A. Preliminaries

1) Notation: In the sequel, we use uppercase and lowercase letters to denote matrices and vectors, respectively. Besides, $|\mathcal{D}|$ represents the total number of elements in set $\mathcal{D}$.

2) Problem Definition: Let $\mathbf{x} \in \mathbb{R}^d$ be a network traffic flow comprising d attributes or dimensions. Consider $\mathbf{x}^h \in \mathbb{R}^d$ and $\mathbf{x}^r \in \mathbb{R}^d$ represent previously and recently collected samples, respectively. $y \in \mathbb{R}^2 : \{0, 1\}$ is the corresponding binary label of $\mathbf{x}$ (i.e., benign or malicious). Given a labeled historical dataset $\mathcal{D}^h = \{(\mathbf{x}_1^h, y_1^h), (\mathbf{x}_2^h, y_2^h), \ldots, (\mathbf{x}_{|\mathcal{D}_h|}^h, y_{|\mathcal{D}_h|}^h)\}$, an unlabeled recent dataset $\mathcal{D}^r = \{\mathbf{x}_1^r, \mathbf{x}_2^r, \ldots, \mathbf{x}_{|\mathcal{D}_r|}^r\}$, and a test dataset $\mathcal{D}^t = \{(\mathbf{x}_1^t, y_1^t), (\mathbf{x}_2^t, y_2^t), \ldots, (\mathbf{x}_{|\mathcal{D}_t|}^t, y_{|\mathcal{D}_t|}^t)\}$. Let $P(\mathbf{x}^h)$ and $P(\mathbf{x}^r)$ represent distributions of $\mathcal{D}^h$ and $\mathcal{D}^r$, respectively, and $\mathcal{D}^t$ shares a similar distribution with $\mathcal{D}^h \cup \mathcal{D}^r$. Under the concept drift problem setting [14], $\mathcal{D}^r$ is considered to drift from $\mathcal{D}^h$, which means $P(\mathbf{x}^h) \neq P(\mathbf{x}^r)$, resulting in the well-trained model on $\mathcal{D}^h$ failing to generalize on $\mathcal{D}^t$. Due to the difference between the two distributions, the goal of concept drift adaptation is to learn a statistical model $\mathcal{M}(\theta) : \mathbb{R}^d \to \mathbb{R}^2$ using all the given data in $\mathcal{D}^h$ and $\mathcal{D}^r$ to minimize the prediction error $\sum_{i=1}^{|\mathcal{D}^t|} |\hat{y}_i^t - y_i^t|$, where $\hat{y}_i^t$ is the predicted label of the ith test instance by the model $\mathcal{M}(\theta)$, and y_i^t is the corresponding ground truth. $\mathcal{M}(\theta)$ can be expressed as $f \circ g$, where f is a feature extractor mapping input $\mathbf{x}$ to its latent representation and g coherently generates binary predictions based on the representation.

3) Problem Scope: In this paper, we primarily focus on concept drift driven by data drift, which refers to the changes in the underlying distribution of data over time [14]. Specifically, as attack techniques evolve, the characteristics of network traffic flows change, leading to a deviation between the distribution of historical data and recent traffic. We examine how this drift affects the performance of intrusion detection models typically trained on historical data. As illustrated in Fig. 2, this drift often results in a decline in model performance, a phenomenon known as model drift [15]. Existing methods often rely on manual labeling, which is costly and time-consuming, especially in security applications. Therefore, we aim to propose a method capable of adapting to concept drift without the need for extensive manual

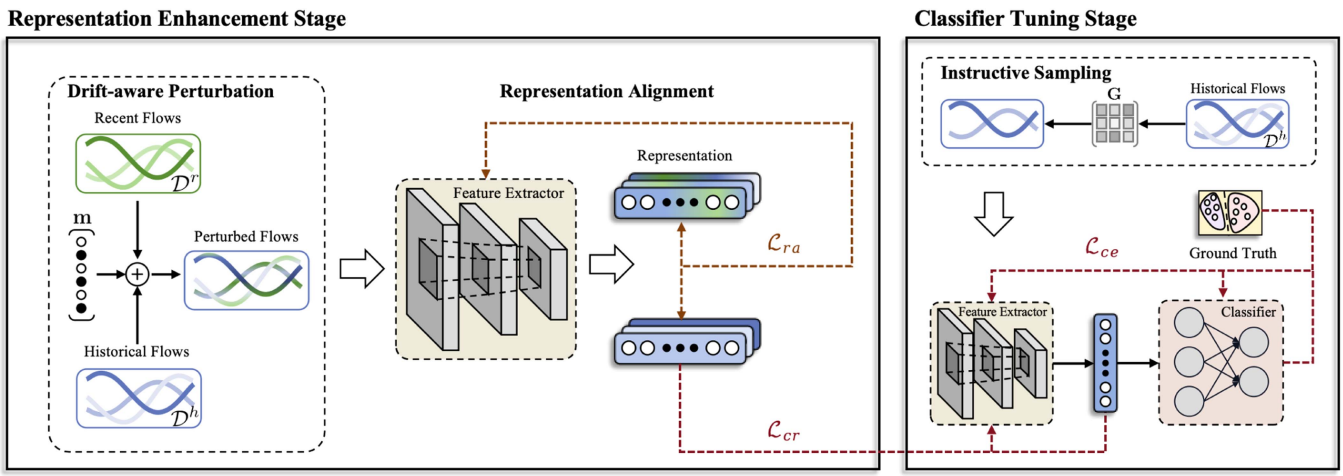


Fig. 3. The overview of proposed ReCDA.

labeling, offering an efficient solution to the challenges posed by data drift in network security.

4) Architecture Overview: We illustrate the proposed ReCDA in Fig. 3, which is a robust concept drift adaptation method with representation enhancement for concept drift without relying on any manual labeling in a network environment subject to the continuous emergence of attack variants. ReCDA consists of two primary stages:

(a) *Self-supervised representation enhancement:* During this stage, carefully designed drift-aware perturbation and representation alignment modules are employed to map traffic flows from the original space to the latent space, ensuring drift-aware and drift-invariant. Here's a detailed explanation of these two properties:

- *Drift-aware:* ReCDA can integrate information from both the historical and drift traffic flows, allowing it to adjust its representation accordingly.
- *Drift-invariant:* ReCDA remains stable and unaffected by changes in underlying data distribution, thus ensuring robustness to concept drift.

(b) *Weakly-supervised classifier tuning:* During this stage, we propose an instructive sample selection module and a classifier fine-tuning module based on representation constraint to efficiently incorporate discriminative knowledge into the model, further enhancing predictive robustness under concept drift scenarios. More technical details are described as follows.

B. Drift-Aware Perturbation

According to the problem definition, concept drift refers to distribution shifting between the historical dataset and the recent dataset. While the intuitive would be to minimize the prediction error by narrowing the gap between the distributions of the historical and drifted data, estimating the distribution of drifted samples is challenging in practice due to constantly emerging attack variants and the risk of model collapse [54] caused by strict distribution consistency. Our proposed method is based on a key observation: for traffic flows, distribution drift corresponds to feature drift in the original space, wherein local changes in feature values drive distribution drift. Therefore, we expect that the model can perceive changes in the features of unlabeled drift flows and capture the relationship between the historical traffic flows and them.

To implement this idea, we propose a drift-aware perturbation method that involves randomly sampling the marginal distribution of features in the drift sample and perturbing the historical sample with a certain probability. Unlike common perturbation methods such as adding random noise [55], [56] or replacing features with meaningless values [37], [57], [58], our approach aims to maintain semantic retention of features while introducing randomness to encourage the model to perceive feature changes in unlabeled drift flows and capture their correlation with the original flow samples.

As shown in Fig. 3, a mask vector $\mathbf{m} = [m_1, m_2, \ldots, m_d]^T$ is generated, where the vector elements $m_k \in \{0, 1\}, k \in [1, d]$ are extracted from a Bernoulli distribution with a certain probability σ. Subsequently, a sample $\mathbf{x}_i^h$ from the historical dataset and the mask vector are used jointly as inputs. Drift-aware perturbation samples are generated as follows:

$$\tilde{\mathbf{x}}_i = \mathbf{m} \odot \mathbf{x}_i^r + (1 - \mathbf{m}) \odot \mathbf{x}_i^h. \tag{1}$$

Using (1), $\tilde{\mathbf{x}}_i$ can be obtained by sampling the feature's marginal distribution of the drift sample. The process bridges the gap between $\mathbf{x}^h$ and $\mathbf{x}^r$, allowing for the smooth elimination of the distribution discrepancies under the control of σ. The parameter σ can be considered a perturbed rate that regulates the degree of fusion between the two distributions. The generated samples gain awareness of drift while retaining the semantic information of the original flow. The randomness introduced by the perturbation increases the diversity of the traffic views, which indicates that our method can better simulate feature drift in real concept drift scenarios.

C. Representation Alignment

Following perturbation, we obtain the drift-aware perspective $\tilde{\mathbf{x}}_i$ of the selected original sample $\mathbf{x}_i^h$. After that, we aim for the model to map drift samples and historical samples to adjacent latent subspaces and learn representations invariant to drift. To achieve this goal, we employ the encoder network as a feature extractor and utilize contrastive learning [59] to encourage the representation of $\tilde{\mathbf{x}}_i$ and $\mathbf{x}_i^h$ to be close. Specifically, for each historical sample $\mathbf{x}_i$ in a mini-batch $\mathcal{B}$, we generate its perturbed view $\tilde{\mathbf{x}}_i$ using (1). Then we feed them to the encoder network f, and the resulting outputs are passed to a projection head h to obtain the corresponding latent representations $\mathbf{z}_i = h \circ f(\mathbf{x}_i)$ and $\tilde{\mathbf{z}}_i = h \circ f(\tilde{\mathbf{x}}_i)$. The contrastive loss can be expressed as:

$$\mathcal{L}_{ra}\left(\mathbf{x}_i\right) = -\log\left(\frac{\exp\left(\mathbf{z}_i \cdot \tilde{\mathbf{z}}_i/\tau\right)}{\sum_{j=1}^{|\mathcal{B}|}\exp\left(\mathbf{z}_i \cdot \tilde{\mathbf{z}}_j/\tau\right)}\right), \tag{2}$$

where $\tau > 0$ is the temperature parameter. Minimizing (2) ensures that $\mathbf{z}_i$ becomes closer to $\tilde{\mathbf{z}}_i$ and be farther from $\tilde{\mathbf{z}}_j$ for $i \neq j$, thereby yielding a more generalized representation with drift-aware and drift-invariant. Thus, the parameter optimization of the encoder in the representation enhancement stage can be formalized as follows:

$$\theta_f^* = \min_{\theta_f}\left[\mathcal{L}_{ra}\left(\mathbf{x}_i|\theta_f\right)\right], \tag{3}$$

where θ_f^* denote updated parameters of encoder network f.

D. Instructive Sampling Strategy

In the preceding stage, although we obtained a drift-robust feature extractor, its direct utilization for detecting malicious activities within the network is impeded by a crucial limitation—it lacks knowledge in distinguishing between benign and malicious network activities. To address this limitation and facilitate the attainment of a robust classifier, we consider using labeled samples from the historical dataset to fine-tune both the feature extractor and classifier. This refinement injects discriminative knowledge into the model, enhancing its ability to discern between benign and malicious network activities. However, incorporating a large number of labeled samples from $\mathcal{D}^h$ during fine-tuning tends to exacerbate the risk of overfitting to the historical distribution $P(\mathbf{x}^h)$, consequently yielding a vanilla classifier that inadequately recognizes drifting traffic flows. To mitigate this issue, we advocate for the selective use of instructive historical instances as the source of discriminative knowledge. Here, we introduce the instructive sampling strategy, delineated by the following two principles:

- Instructive samples should be distant from drift samples in original space.
- Instructive samples should resemble the representation of the drift sample in latent space.

The original space encompasses samples directly extracted from the dataset, while the latent space denotes the embedding space of the feature extractor output. These principles guide the selection process, ensuring that the chosen instructive samples possess characteristics conducive to model training and drift adaptation. The degradation of the model caused by concept drift can generally be attributed to the classifier's challenge in effectively detecting attacks based on the representations generated by the feature extractor. Specifically, the features of recently arrived malicious samples drift, exhibiting confusion with benign samples. Distinguishing these samples in the original space is challenging because they correspond to similar feature values and are usually assigned to the same category. Hence, our intuition is to select labeled historical samples located at the potential decision boundary, as their representations contain richer category-discriminative knowledge. To facilitate the model in capturing the generalized difference between benign and malicious activities under the guidance of such a sub-dataset, we introduce the instructive matrix $G \in \mathbb{R}^{|\mathcal{D}^h|\times 2}$ for determining the selection of samples. It is defined as follows:

$$G_{i,j} = \frac{sim(\mathbf{x}_i^h, \bar{\mathbf{x}}_j^r|\bar{\mathbf{y}}_j)}{sim(f(\mathbf{x}_i^h), f(\bar{\mathbf{x}}_j^r)|\bar{\mathbf{y}}_j) + \epsilon}. \tag{4}$$

Here, $sim(\cdot)$ represents the distance measurement function, with the cosine distance being used due to its effectiveness in measuring the similarity between high-dimensional vectors. $\bar{\mathbf{x}}_j^r$ denotes clustering centroids of samples belonging to the category $\bar{\mathbf{y}}_j$ in the original space. Specifically, binary clustering is performed on the unlabeled recent dataset to obtain the clustering centroids involved in the distance calculation, which accelerates computation and reduces storage requirements. Additionally, an extra small value ϵ is added to the denominator for numerical stability.

According to (4), higher instructive metrics indicate more influential samples in constructing the decision boundary. Hence, the top-δ historical samples will be selected to participate in the constrained classifier tuning phase.

E. Constrained Classifier Tuning

To train a classifier for network intrusion detection, we integrate a classification head g to the encoder network f, which takes the output of f as the input and predicts the label of the instance. During the classifier tuning stage, we expect that the robust feature extractor that has been generated will not lose the valuable knowledge that has been gained. Therefore, we constrain the variation of the representation for subsequent learning to maintain the effectiveness of the robust feature extractor. The consistency regularization loss is defined as:

$$\mathcal{L}_{cr}(\mathbf{x}_i) = \left\| f(\mathbf{x}_i|\theta_f) - f(\mathbf{x}_i|\theta_f^*)\right\|^2. \tag{5}$$

$\mathcal{L}_{cr}$ aims to encourage the tuned feature extractor $f(\theta_f^*)$ to return a similar output distribution learned during the representation enhancement stage. In addition, we employ cross-entropy loss to guide the classifier in acquiring category-discriminative knowledge. Let $\mathcal{L}_{ce}(\mathbf{x}_i, y_i)$ represent the supervised loss; the optimization of the feature extractor and classifier in the fine-tuning stage are formalized as follows:

$$\theta_g^*, \theta_f^* = \min_{\theta_g,\theta_f}\left[\mathcal{L}_{ce}\left(\mathbf{x}_i, y_i|\theta_g, \theta_f\right) + \lambda\mathcal{L}_{cr}\left(\mathbf{x}_i|\theta_f\right)\right]. \tag{6}$$

Here, θ_g^* denotes updated parameters of classifier g, and λ is a balanced coefficient to control the regularization strength.

Algorithm 1: Algorithm of ReCDA.

Input: historical dataset $\mathcal{D}^h$, recent dataset $\mathcal{D}^r$, batch size $|\mathcal{B}|$, perturbation rate σ, temperature τ, constant ϵ, coefficient λ, encoder network f, projection head h, classifier g.

Output: encoder network f and classifier g

let d be the dimension of input samples.
let θ and θ^* be model parameters before and after one iteration.
for sampled mini-batch $\{\mathbf{x}_i^h\}_{i=1}^{|\mathcal{B}|} \subseteq \mathcal{D}^h$ **do**
  for $i = 1, \ldots, |\mathcal{B}|$ **do**
    generate $\mathbf{m} \in \mathbb{R}^d$ where $m_k \in [0, 1]$ and $p(m_k = 1) = \sigma$.
    sample $\mathbf{x}_i^r$ from $\mathcal{D}^r$.
    let $\tilde{\mathbf{x}}_i = \mathbf{m} \odot \mathbf{x}_i^r + (1 - \mathbf{m}) \odot \mathbf{x}_i^h$.
    let $\mathbf{z}_i = h \circ f(\mathbf{x}_i^h)$, $\tilde{\mathbf{z}}_i = h \circ f(\tilde{\mathbf{x}}_i)$.
  end for
  let $\mathcal{L}_{ra} := -\frac{1}{|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} \log(\frac{\exp(\mathbf{z}_i \cdot \tilde{\mathbf{z}}_i/\tau)}{\sum_{j=1}^{|\mathcal{B}|} \exp(\mathbf{z}_i \cdot \tilde{\mathbf{z}}_j/\tau)})$.
  update f and h to minimize $\mathcal{L}_{ra}$.
end for
cluster $\mathcal{D}^r$ to get centroids $\bar{\mathbf{y}} \in [0, 1]$.
generate $G \in \mathbb{R}^{|\mathcal{D}^h| \times 2}$, $G_{i,j} = \frac{sim(\mathbf{x}_i^h, \bar{\mathbf{x}}_j^r | \bar{\mathbf{y}}_j)}{sim(f(\mathbf{x}_i^h), f(\bar{\mathbf{x}}_j^r) | \bar{\mathbf{y}}_j) + \epsilon}$.
select top-n samples from $\mathcal{D}^h$ according G to generate sub-dataset $\mathcal{D}_{sub}^h$.
for sampled mini-batch $\{\mathbf{x}_i^h\}_{i=1}^{|\mathcal{B}|} \subseteq \mathcal{D}_{sub}^h$ **do**
  let $\mathcal{L}_{cr} := \frac{1}{|\mathcal{B}|} \sum_{i=1}^{|\mathcal{B}|} \|f(\mathbf{x}_i|\theta_f) - f(\mathbf{x}_i|\theta_f^*)\|^2$.
  update f and g to minimize $\mathcal{L}_{ce} + \lambda \mathcal{L}_{cr}$.
end for

By optimizing (6), the model obtains category-discriminative knowledge while preserving the drift awareness and drift invariance representation obtained during the representation enhancement stage. This results in a generalized model capable of robust concept drift adaptation. The algorithm of ReCDA is shown in Algorithm 1.

IV. Experiments

This section outlines evaluation settings, which provide a more challenging evaluation compared with common settings. Following the settings, we conduct extensive comparative experiments to analyze the effectiveness and robustness of the proposed concept drift adaptation method against existing approaches.

A. Evaluation Setting

1) Offline Evaluation: Previous studies [17], [43], [44] predominantly conducted experiments under the one-vs-rest setting, wherein only one type of attack is deemed as drift. However, we observe that this evaluation approach is conservative and often yields an overly optimistic assessment of performance. Furthermore, it is important to note that scenarios where a previously unseen attack type is introduced during evaluation—such as in some of our defined drift cases—are technically more aligned with concept evolution [60] (i.e., the emergence of a new sub-class) rather than concept drift, which typically involves changes in the distribution of known classes. Nevertheless, since our system is designed for binary classification (benign vs. malicious), the introduction of a new attack type within the broader "malicious" class effectively results in a shift of the underlying data distribution from the model's perspective. Therefore, we include such scenarios under the umbrella of concept drift to comprehensively evaluate model adaptivity and robustness. To address the limitations of previous evaluation protocols, we advocate for a more rigorous approach by simultaneously considering multiple attack categories as drift data, thereby substantially intensifying the concept drift (or evolution) encountered by the model. Additionally, evaluating model stability across varying degrees of drift is challenging when using a non-uniform test dataset. To ensure a fair evaluation, we propose adhering to the original dataset partition or adopting a fixed training and test set split method. We employ two widely used network intrusion detection datasets to simulate realistic drift scenarios for offline evaluation. The details are as follows.

TABLE II
The Statistics of UNSW-NB15 Dataset

Category		Training Set	Test Set
Benign		56000	37000
DoS		12264	4089
Exploits		33393	11132
Fuzzers		18184	6062
Generic		40000	18871
Reconnaissance		10491	3496
Others	Analysis	2000	677
	Backdoors	1746	583
	Worms	130	44
	Shellcode	1133	378
Total		175341	82332

UNSW-NB15 dataset: The raw network packets of the UNSW-NB15 dataset [61] were collected by the IXIA PerfectStorm tool in the Cyber Range Lab at the University of New South Wales (UNSW), Australia. The dataset is designed to simulate real-world network traffic and includes a diverse range of network traffic features, encompassing both benign and nine malicious activities, namely, DoS, Exploits, Fuzzers, Generic, Reconnaissance, Analysis, Backdoors, Worms, and Shellcode. For reliable evaluation, we maintain the original dataset split: 175,341 training data instances and 82,332 testing data instances, in which the test set contains all nine types of attacks. Given that the drift of the less prevalent attack categories has minimal impact on the evaluation, we merge the four attacks denoted as "Others", the statistics of categories in the training and test datasets are shown in Table II. Additionally, each data point consists of 44 features. We apply a commonly used feature engineering method, where categorical features are represented using label encoding and numerical features are scaled by z-score normalization.

To evaluate our method, we iteratively select samples from different sets of malicious categories to serve as drift data. In this way, we split the original training set into various combinations

TABLE III
THE STATISTICS OF CICIDS-2017 DATASET

Day	Benign	Malicious	Total	# of Categories
Monday	371689	0	371689	1
Tuesday	315008	6953	321961	4
Wednesday	319186	171790	490976	6
Thursday	359973	222	360195	5
Friday	291213	254884	546097	4

of historical dataset $\mathcal{D}^h$ and recent dataset $\mathcal{D}^r$, with $\mathcal{D}^h$ being labeled and $\mathcal{D}^r$ unlabeled. Here, we define the drift index to quantify the degree of drift:

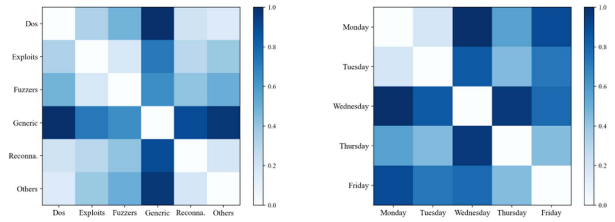
$$C\#i : i = |L\left(\mathcal{D}^h \cup \mathcal{D}^r\right) - L(\mathcal{D}^h)|, \tag{7}$$

where $L(\mathcal{D}^h)$ and $L(\mathcal{D}^r)$ represent the number of categories in the historical dataset and recent dataset, respectively. For example, C#2 indicates that the $\mathcal{D}^h$ comprises four categories of malicious activity, while the remaining two types appear in the $\mathcal{D}^r$. For any i, there are $C_6^i = \frac{6!}{i!(6-i)!}$ data partitioning cases, which can be viewed as a combination problem. For better comparison, the reported results in the sequel are averaged across all data partitioning cases.

CICIDS-2017 dataset: The CICIDS-2017 dataset, developed by the Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick, is a comprehensive and widely used benchmark dataset for network intrusion detection research. Following the recommendation in INSOMINA [26], we use the refined version of CICIDS-2017 [62] and the statistics of the dataset are shown in Table III. The new version reconstructed or relabelled 20 percent of the original traffic traces to rectify issues related to traffic generation, flow construction, feature extraction, and labeling. Consequently, the dataset comprises 2,524,767 timestamped flows and 72 features, covering 5 days and encompassing 15 malicious categories, including brute force attacks, Heartbleed, botnet communication, DoS and DDoS variants, infiltration, and web-related threats.

CICIDS-2017 provides timestamps for traffic flows so we can obtain drift data more consistently with real-world scenarios. Initially, we partition each day's data into the training and test sets with a ratio of 8:2. The entire traffic flow sequence in the training set can be formalized as $\mathcal{D} = (\mathcal{X}_t, Y_t) : t = 1, 2, 3, 4, 5$, where $\mathcal{X}_t$ represent the traffic trace collected in the tth day, Y_t is the corresponding label of $\mathcal{X}_t$. Then we divide the $\mathcal{D}$ into the historical dataset $\mathcal{D}^h = (\mathcal{X}_1, Y_1), \ldots, (\mathcal{X}_{i+1}, Y_{i+1})$ and recent dataset $\mathcal{D}^r = (\mathcal{X}_{i+2}, Y_{i+2}), \ldots, (\mathcal{X}_5, Y_5)$. Time-based splitting implies that the recent dataset contains sample features absent from the historical dataset, thus inducing concept drift. Accordingly, the drift index can be defined as T@i, where for $t > i + 1$, the label of traffic instance is not available in the training phase.

We use the Optimal Transport Dataset Distance (OTDD) [63] to explore the distribution differences over categories or timestamps. OTDD is a geometric method for calculating distances between probability distributions to compare datasets. For better illustration, a min-max scale is applied to the calculated distance matrix. As depicted in Fig. 4, whether the dataset is split over categories or timestamps, it introduces varying degrees of drift



(a) UNSW-NB15 (b) CICIDS-2017

Fig. 4. Optimal transport dataset distance for UNSW-NB15 dataset and CICIDS-2017 dataset.

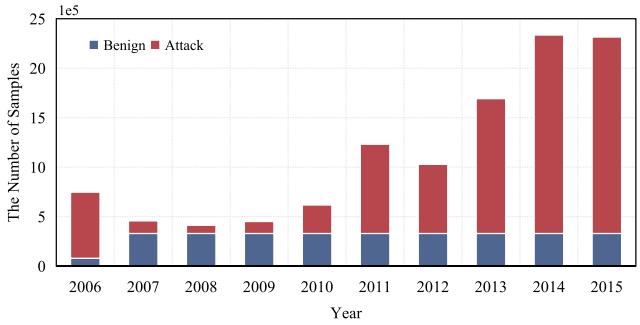


Fig. 5. The statistics of kyoto-2006+ dataset.

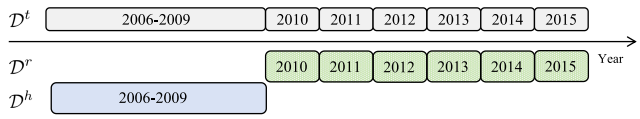


Fig. 6. The splits over kyoto-2006+ dataset for online evaluation.

into the evaluation. For example, the distribution of flows collected on Wednesday in the CICIDS-2017 dataset exhibits more significant differences compared to other days, posing a greater challenge for adapting to concept drift in the case of T@1.

2) Online Evaluation: In this section, we take ReCDA as a plug-and-play representation extraction module and evaluate its performance in the online environment through the continuous learning framework. The model will be updated as time passes. Considering the data volume limitations of UNSW-NB15 and CICIDS-2017, we introduce the refined Kyoto-2006+ dataset [64] recently released for concept drift evaluation. The statistics of the dataset are shown in Fig. 5.

Kyoto-2006+ dataset: The Kyoto-2006+ dataset is built on 10 years of real traffic data (Nov. 2006 - Dec. 2015), captured by a system of 348 honeypots in 5 sub-networks inside the Kyoto University. The original version includes 2 categorical features and 12 numerical features, with naturally occurring changes over time (e.g., users modifying their behavior patterns, and software updates). Dragoi et al. [64] conducted a thorough analysis from multiple angles and refined it to develop models that generalize better and are more robust to drifts in data.

As shown in Fig. 6, we apply a chronology-based evaluation protocol for building historical and recent splits that can highlight the temporal evolution of data. The split settings follow the

suggestion of [64] and consider the overall distribution distances between years. The historical dataset $\mathcal{D}^h$ is extracted from the first period, and the test will show the expected performance when there is no distribution shift between the training and test datasets $\mathcal{D}^t$. Each year after 2010 will be regarded as the recently arrived dataset $\mathcal{D}^r$, and some flows will be separated as a test dataset for independent evaluation. A larger year indicates a greater degree of drift, and therefore a more significant expected decline in performance.

In summary, under our evaluation protocol, the entire dataset is divided into a historical dataset, denoted as $\mathcal{D}^h$, and a recently arrived dataset, $\mathcal{D}^r$, following the specific partitioning strategies outlined in the previous section. The historical dataset is labeled, while the recently arrived dataset remains unlabeled. During the representation enhancement stage, both $\mathcal{D}^h$ and $\mathcal{D}^r$ are utilized for self-supervised training of the encoder, with the recently arrived dataset serving as the source of perturbations. For each mini-batch of historical data with a batch size of $\mathcal{B}$, we randomly select $\mathcal{B}$ samples from $\mathcal{D}^r$ as sources of perturbation. Consequently, the number of perturbed samples in each mini-batch matches the number of original historical samples, ensuring consistency in representation alignment for each historical instance and its perturbed view. In the classifier tuning stage, the model has access to the labels of the historical dataset $\mathcal{D}^h$, enabling it to incorporate discriminative knowledge effectively. For the testing phase, we implement different strategies for offline and online evaluations. In offline evaluation, all settings utilize the same test dataset to ensure comparability. In online evaluation, the test dataset is derived from traffic collected over the most recent year for each dataset split, allowing us to evaluate performance in real-world scenarios.

B. Experimental Setup

We present the experimental setup in this subsection, including baseline methods, evaluation metrics, and parameter settings.

1) Baseline Methods: We compare the proposed method against three common machine learning methods: Logistic Regression (LR), K-Nearest Neighbors (KNN) [65], and Decision Tree (DT) [66]. We also include the latest method [36], [37], [38] for network traffic classification and two state-of-the-art concept drift adaptation methods [17], [26] for network intrusion detection. Here is a brief introduction to them.

LEXNet [36] is a lightweight, efficient, and explainable convolutional neural network designed for network traffic classification. It relies on a new residual block and prototype layer and shows superior performance on a commercial-grade dataset.

CLEID [37] is an intrusion detection framework based on contrastive learning, which utilizes a heuristic method to construct sample pairs based on random masking effectively. Semantic relationships among different samples are extracted to enhance the robustness of the model.

ACID [38] is a classifier-agnostic and highly effective intrusion detection system. It introduces supervised adaptive clustering techniques to learn cluster centers that can be used as extensions of the input features, which improves the robustness against outliers and the generalization abilities of detection models. We followed their disclosed implementation, i.e., feed-forward networks are employed.

CADE [17] is a representative method for adapting to concept drift by labeling the detected drifting samples. Specifically, CADE maps the data samples into a low-dimensional space and learns a distance function to measure dissimilarity between samples. To ensure a fair comparison, we followed their disclosure setting and adapted the implementation to binary classification. It should be noted that CADE is primarily designed for drift detection, so we calculate the union of correct drift detection and correct classification to evaluate its performance. In other words, for a drift sample, as long as CADE can detect its drift, even if the classification result is wrong, it still contributes to final accuracy.

INSOMNIA [26] is a semi-supervised intrusion detector that updates the base model as network traffic characteristics are affected by concept drift. They also consider the cost of labeling, so in the model update phase, they use Nearest Centroid (NC)-based strategies to generate pseudo-labels for drift adaptation. It is worth noting that our focus is on robust concept drift adaptation methods aimed at achieving good generalization on drift traffic flows, rather than continuous incremental learning. Therefore, the base model of INSOMNIA is well trained on $\mathcal{D}^h$, and $\mathcal{D}^r$ is provided to INSOMNIA for model update at once.

TRIDENT [50] is a unified framework designed for both concept drift detection and adaptation. It transforms the fine-grained classification problem of intrusion detection into multiple independent one-class learning tasks, using Extreme Value Theory to determine boundary thresholds. When new classes emerge, the framework employs an incremental learning strategy, utilizing ensemble clustering to identify unknown traffic and adapting to concept drift through the lateral expansion of classifiers. In our experiment, we evaluated TRIDENT(AE) and performed incremental updates using $\mathcal{D}^r$.

2) Evaluation Metrics: We use Accuracy and F1-score to evaluate the performance of the proposed method and its competitors. Accuracy measures the overall correctness of the model, while the F1-score provides a balanced assessment of both precision and recall, calculated as $F1-score = \frac{2\times precision\times recall}{precision+recall}$. These metrics offer a comprehensive evaluation of the model's ability to correctly identify both positive and negative classes, particularly in the presence of imbalanced data.

3) Parameter Settings: In the representation enhancement stage, we use two fully connected layers as the feature extractor, followed by a linear head consisting of a fully connected layer. The hidden layer size is set to 16 and 64 for offline evaluation and online evaluation, respectively. In the classifier tuning stage, we initialize a single fully connected layer as a classifier to perform binary classification, i.e., distinguish between benign and malicious traffic, which has the same input dimension as the output of the feature extractor. Both stages are trained with Adam optimizer using the initial learning rate of 0.0001 and batch size of 128. The representation model and classifier model are trained for 100 epochs and 50 epochs, respectively. All experiments are run on NVIDIA 4090 GPUs for fair comparisons.

TABLE IV
COMPARISONS ON UNSW-NB15 DATASET

Method	C#1		C#2		C#3		C#4		C#5	
	ACC	F1	ACC	F1	ACC	F1	ACC	F1	ACC	F1
LR	0.81±0.03	0.80±0.04	0.80±0.06	0.79±0.06	0.78±0.08	0.78±0.08	0.75±0.09	0.74±0.09	0.69±0.09	0.68±0.10
KNN	0.84±0.03	0.84±0.04	0.83±0.05	0.83±0.05	0.81±0.09	0.80±0.09	0.77±0.07	0.77±0.07	0.70±0.07	0.69±0.07
DT	0.87±0.03	0.87±0.03	0.87±0.05	0.87±0.05	0.84±0.10	0.84±0.10	0.81±0.11	0.81±0.12	0.72±0.13	0.71±0.14
LEXNet	0.86±0.05	0.88±0.04	0.86±0.06	0.87±0.07	0.81±0.09	0.83±0.08	0.74±0.14	0.67±0.29	0.63±0.12	0.46±0.26
ACID	0.86±0.03	0.85±0.03	0.84±0.06	0.84±0.06	0.82±0.07	0.81±0.07	0.78±0.07	0.78±0.07	0.72±0.06	0.71±0.06
CLEID	0.87±0.02	0.87±0.02	0.87±0.03	0.87±0.03	0.85±0.05	0.84±0.05	0.81±0.06	0.81±0.06	0.73±0.09	0.72±0.09
INSOMNIA	0.88±0.05	0.86±0.05	0.85±0.06	0.84±0.06	0.82±0.07	0.80±0.09	0.75±0.09	0.70±0.15	0.70±0.11	0.52±0.28
CADE	0.84±0.05	0.88±0.04	0.82±0.05	0.86±0.03	0.79±0.03	0.84±0.02	0.80±0.03	0.84±0.02	0.79±0.03	0.84±0.02
TRIDENT	0.84±0.02	0.83±0.03	0.82±0.06	0.82±0.06	0.82±0.03	0.81±0.04	0.81±0.06	0.80±0.06	0.80±0.03	0.80±0.04
ReCDA	**0.91±0.01**	**0.91±0.01**	**0.91±0.03**	**0.90±0.03**	**0.89±0.04**	**0.89±0.04**	**0.87±0.06**	**0.87±0.06**	**0.86±0.05**	**0.86±0.04**

C. *Offline Experiment Results*

In this section, we first compare the overall performance of ReCDA with other baselines and evaluate the robustness of the model to drift adaptation under varying degrees of drift. Subsequently, we delve into a more detailed investigation under a fixed drift index.

1) Overall Evaluation: In Table IV, we report the experimental results on the UNSW-NB15 dataset. As described in Section IV-A1, for drift index C#i, we iteratively considered i type(s) of attacks among the 6 malicious categories as drift data. The results presented are the averages and standard deviations under all category combinations. From the results in Table IV, it is evident that our method exhibits excellent adaptability and stability in handling concept drift across different degrees of drift. Particularly in the case of extreme drift (C#5), our method maintains an accuracy that is 7% higher than that of the sub-optimal method.

For all considered machine learning models and LEXNet, we observe performance degradation as the degree of drift increases, emphasizing the inability of these intrusion detection models to cope with changes in data distribution resulting from concept drift. For example, the accuracy of LEXNet drops from 0.86 at C#1 to 0.63 at C#5, performing even worse than LR. This indicates that while the elaborated DL-based model fits the historical dataset well, it struggles to generalize in concept drift scenarios. Regarding ACID and CLEID designed to improve the generalization ability of intrusion detectors, the results show their limited drift adaptability, which is vulnerable under more rigorous evaluation settings.

C#1 is widely used as the benchmark setting for concept drift, i.e., only a type of attack is unknown in the training phase. However, we have found that this setting tends to give an illusion of high performance, as the performance deteriorates rapidly with an increase in the degree of drift. For example, INSOMNIA reaches a competitive accuracy of 0.88 at C#1 but suffers from significant performance degradation by 6% and 18% when shifted to C#3 and C#5, accompanied by a steep increase in standard deviation. Recall that INSOMNIA applies NC-based strategies to generate pseudo-labels for drift adaptation. The experiment results show that the estimation of pseudo-label is impractical when lack of sufficient neighbor samples, and the performance of the model is easily affected by the label noise, which leads to the phenomenon of self-poisoning. TRIDENT also employs clustering for label assignment, but it utilizes different parameters and integrates the results to achieve relatively stable outcomes. This approach reduces the risk of self-poisoning to some extent. However, the overall performance of the method is not well due to the lack of comparative information between classes in the one-class classifiers. In comparison to other methods, ReCDA demonstrates greater resilience to increasing degrees of drift, with ReCDA exhibiting an accuracy 10% and 7% higher than that of CADE at C#3 and C#5, respectively.

TABLE V
COMPARISONS ON CICIDS-2017 DATASET

Method	T@1		T@2		T@3	
	ACC	F1	ACC	F1	ACC	F1
LR	0.596	0.577	0.761	0.747	0.768	0.756
KNN	0.641	0.615	0.814	0.808	0.778	0.773
DT	0.508	0.351	0.710	0.684	0.711	0.685
LEXNet	0.511	0.674	0.808	0.832	0.761	0.783
ACID	0.652	0.619	0.814	0.808	0.814	0.813
CLEID	0.640	0.605	0.816	0.809	0.814	0.808
INSOMNIA	0.854	0.851	0.808	0.767	0.808	0.768
CADE	0.850	0.835	0.952	0.950	0.963	0.961
TRIDENT	0.775	0.769	0.845	0.839	0.825	0.853
ReCDA	**0.872**	**0.872**	**0.954**	**0.954**	**0.963**	**0.963**

In practical applications, dynamic changes in network environments and user behaviors lead to concept drift in traffic flows, which subsequently deteriorates the performance of intrusion detection systems over time. To evaluate the adaptability of models to this drift, we partitioned the CICIDS-2017 dataset based on timestamps. As shown in the experimental results in Table V, ReCDA achieves superior performance across all scenarios. In contrast, TRIDENT exhibits poor performance in this setting. The simultaneous input of both benign and malicious traffic complicates the classifier updates and adaptation processes, as they must handle both types of samples within the new categories affected by normality drift [49]. While CADE closely approximates the performance of our method at T@2 and

TABLE VI
ONLINE EVALUATION ON KYOTO-2006+ DATASET

Method	2006-2009		2010		2011		2012		2013		2014		2015	
	ACC	F1	ACC	F1	ACC	F1	ACC	F1	ACC	F1	ACC	F1	ACC	F1
LR	0.931	0.923	0.908	0.908	0.945	0.945	0.726	0.734	0.744	0.769	0.675	0.704	0.727	0.734
ReCDA+LR	-	-	0.946	0.954	0.954	0.952	0.954	0.952	0.908	0.866	0.858	0.834	0.799	0.773
KNN	0.944	0.932	0.835	0.833	0.761	0.775	0.827	0.827	0.820	0.835	0.697	0.729	0.699	0.736
ReCDA+KNN	-	-	0.952	0.953	0.957	0.960	0.954	0.953	0.920	0.877	0.855	0.814	0.795	0.762
DT	0.969	0.964	0.949	0.946	0.962	0.962	0.821	0.826	0.821	0.835	0.667	0.701	0.713	0.733
ReCDA+DT	-	-	0.936	0.934	0.949	0.948	0.941	0.940	0.930	0.888	0.852	0.812	0.745	0.761

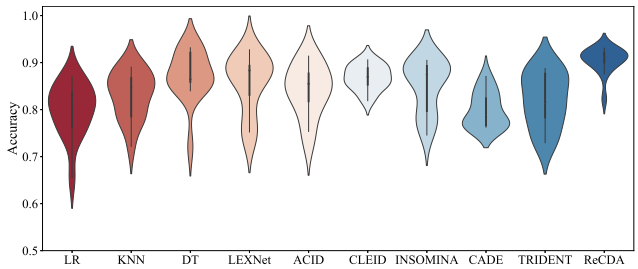


Fig. 7. The detailed results at C#2 on UNSW-NB15 dataset.

T@3, it is important to note that, as mentioned in Section IV-B1, we have adopted a tolerant measure for CADE.

Additionally, for T@1, the performances of DT and LEXNet are approximately equivalent to random guessing, with 0.511 and 0.508 of accuracy, respectively. Despite having parameters and complexity that far exceed baseline models like LR or KNN, LEXNet, as an intrusion detector, exhibits extremely poor generalization performance on concept drift data. When the distribution in the historical dataset is uncorrelated with that in the drift dataset, the model tends to learn specific biases from the historical dataset, making generalization and adaptation to concept drift challenging.

Overall, the experimental results reveal the rapid degradation of traditional models as the degree of drift increases, underscoring their unreliability. Furthermore, these experiments also highlight the issues of the current concept drift adaptation methods and demonstrate the robustness of our proposed approach under varying degrees of drift.

2) Detailed Evaluation: We take a closer look at the detailed performance at C#2 on the UNSW-NB15 dataset. The results depicted in Fig. 7 reveal that various combinations of malicious activities exert different degrees of impact on the intrusion detection model, a phenomenon linked to the similarity of their feature distributions. As shown in Fig. 4(a), certain types of attacks exhibit similar distances, enabling the model to generalize effectively to other categories even when labels for one category are unavailable during training. Nevertheless, our method consistently achieves high accuracy and outperforms the other methods across all cases. Moreover, the area occupied by ReCDA is notably smaller than that of its competitors, indicating that our method exhibits consistency at various degrees of drift. This superior property shows that ReCDA has strong adaptability and robustness.

D. Online Experiment Results

In this section, we first evaluate three common classifiers in online settings and then consider ReCDA as a plug-and-play module to assess its enhancement effect on simple classifiers. From the results shown in Table VI, the following observations and conclusions can be drawn.

- In the online evaluation settings, the performance of simple classifiers shows a declining trend over time. From 2009 (no drift) to 2015, the detection accuracy of LR, KNN, and DT decreased by 21.9%, 26.0%, and 26.4%, respectively. This indicates that models fitted on historical data distributions struggle to generalize to recently arriving data distributions.
- For each classifier, the integration of ReCDA significantly alleviates the issue of performance decline. As an efficient representation enhancement method, ReCDA enables drift adaptation without requiring labels for the recently arriving traffic, allowing the model to maintain robust performance even in the fourth year (with only a 2.5% performance drop for LR and KNN).
- We observe that the enhancement effect of ReCDA on a tree-based model is not as pronounced as that on LR and KNN. It could be attributed to ReCDA mapping high-dimensional raw features to a lower-dimensional space, where the embedded features may not provide sufficient information gain. Therefore, the tree-based model may not be deep enough to capture the complexity of the data.

Furthermore, we plotted the confusion matrices of various methods to evaluate the performance gains of ReCDA across different classifiers. As illustrated in Fig. 8, where 0 represents benign traffic and 1 denotes malicious traffic. It is evident from Fig. 8(a)–(c) that classifiers fitted to historical patterns struggled to detect recently arrived malicious traffic in 2012, leading to increased false negative rates. In contrast, classifiers updated under the guidance of ReCDA effectively adapted to the concept drift, resulting in a significant improvement in classification performance. Therefore, we can conclude that ReCDA enhances the resilience of various classifiers against concept drift.

V. FURTHER ANALYSIS

In this section, we conduct further analysis of ReCDA at T@2 and C#1, where Generic attack in UNSW-NB15 is regarded as drift.

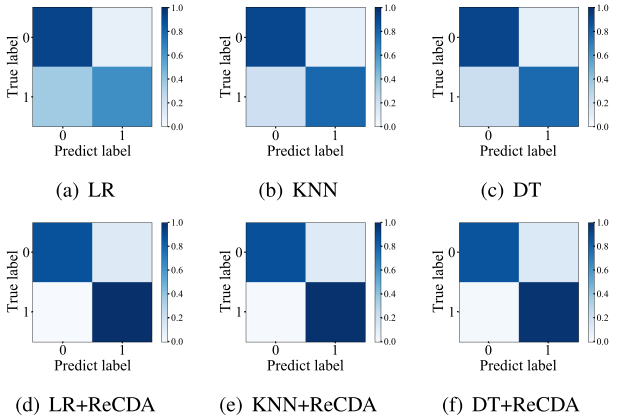


Fig. 8. Confusion matrix at 2012 on KYOTO-2006+ dataset.

TABLE VII
THE INFLUENCE OF PERTURBATION RATE

σ	0.3	0.4	0.5	0.6	0.7	0.8
UNSW-NB15	0.819	0.845	0.839	**0.847**	0.831	0.813
CICIDS-2017	0.833	0.854	0.876	0.878	**0.897**	0.849

TABLE VIII
THE INFLUENCE OF δ AND λ ON UNSW-NB15 DATASET

λ \ δ	25%	50%	75%	100%
0.1	0.893	0.901	0.888	0.887
0.5	0.881	0.899	**0.913**	0.908
0.8	0.869	0.885	0.885	0.898
1.0	0.850	0.866	0.874	0.876

TABLE IX
THE INFLUENCE OF δ AND λ ON CICIDS-2017 DATASET

λ \ δ	25%	50%	75%	100%
0.1	0.856	0.854	0.855	0.853
0.5	0.880	0.893	0.893	0.893
0.8	0.943	**0.954**	0.950	0.943
1.0	0.907	0.918	0.913	0.898

TABLE X
ABLATION STUDY OF RECDA

Method	UNSW-NB15		CICIDS-2017	
	ACC	F1	ACC	F1
ReCDA	**0.913**	**0.911**	**0.954**	**0.954**
ReCDA w/o IS	0.908	0.906	0.943	0.943
ReCDA w/o IS, CT	0.895	0.869	0.849	0.848
ReCDA w/o IS, CT, RA	0.751	0.716	0.761	0.747

A. Parameter Study

We investigate the influence of hyperparameters by varying their values. Our experiment approach involves fixing one parameter while altering the other.

1) The Influence of σ: To assess the impact of perturbation rate σ, we followed the settings outlined in Section IV-B3, except for employing a frozen encoder during the classifier tuning stage. We vary σ from 0.3 to 0.8. As shown in Table VII, the optimal performance on UNSW-NB15 dataset is attained when $\sigma = 0.6$, while for CICIDS-2017 dataset, the value is $\sigma = 0.7$. σ serves as the control parameter determining the fusion degree of historical and drift samples, where the contrastive loss evaluates the similarity between the original flow and its perturbed version. Therefore, excessively large values of σ may destroy the semantics of traffic flows, whereas overly small values may lead to insufficient awareness of concept drift.

2) The Influence of δ and λ: The classifier tuning stage includes two hyper-parameters, the instructive sample selection rate δ, and a factor λ to regulate the regularization strength.

We respectively tune the two hyper-parameters in a range of $\delta \in \{25\%, 50\%, 75\%, 100\%\}$ and $\lambda \in \{0.1, 0.5, 0.8, 1.0\}$. The results are illustrated in Tables VIII and IX. According to the results, the accuracy fluctuates with the change of δ. A large δ may lead to overfitting on the historical dataset, diminishing awareness of drift data, whereas a smaller δ may not furnish the classifier with sufficient discriminative insights. Selection of λ is dataset-dependent; for example, UNSW-NB15 favors a smaller value ($\lambda = 0.5$), whereas CICIDS-2017 shows preference for a larger one ($\lambda = 0.8$).

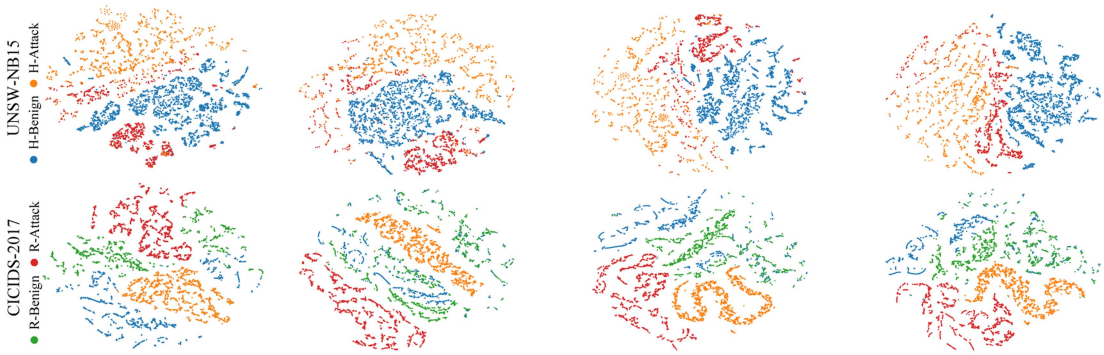
B. Ablation Study

In this section, we conduct an ablation study to analyze the performance gain of each component in ReCDA by removing them gradually. RA, IS, and CT are the abbreviations of representation alignment, instructive sampling, and constrained tuning, respectively. The results are shown in Table X.

The results consistently demonstrate that ReCDA outperforms its variants. Each component in ReCDA contributes to enhancing the predictive model's performance, with the optimal performance achieved when they collaborate in our unified framework. We note that the removal of CT leads to a more significant performance drop in the CICIDS-2017 dataset compared to the UNSW-NB15 dataset. This difference can be attributed to the more severe data shift in CICIDS-2017, resulting in the feature extractor and predictive model being prone to overfitting historical traffic flows while losing awareness of the drift flows. This observation aligns with the findings in Section V-A2. In addition, we observe that the proposed representation alignment significantly enhances the deep representations, and the following instructive sampling strategy further improves the classification performance, which justifies our claims.

C. Complexity Analysis

The time complexity of ReCDA can be analyzed by considering each of its key stages: Representation Enhancement and Classifier Tuning. In the first stage, drift-aware perturbation is applied to each sample individually, with a time complexity of $\mathcal{O}(n_h \cdot d)$. where n_h is the number of historical samples and d is the feature dimension. To align the representations of the original and perturbed data, the algorithm calculates pairwise similarities, which has a time complexity of $\mathcal{O}(n_h^2 \cdot d)$. In practice, mini-batch training reduces this cost to approximately linear



(a) Orginal Space (b) Latent Space(Baseline) (c) Latent Space(Directly) (d) Latent Space(ReCDA)

Fig. 9. t-SNE visualizations on UNSW-NB15 and CICIDS-2017 dataset. We sample part of the traffic flows from $\mathcal{D}^h$ and $\mathcal{D}^r$, denoted H- and R-, respectively. (a) We visualize the samples in the original space. (b) We train a 3-layer MLP as a baseline. (c) We directly generate the representation using the feature extractor of ReCDA. (d) We use the entire ReCDA.

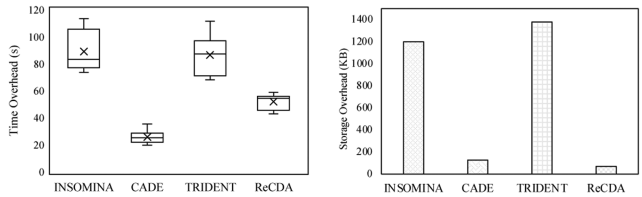
with respect to the batch size. In the second stage, Instructive Sampling involves clustering the recent samples $\mathcal{D}^r$, which has a time complexity of $\mathcal{O}(n_r \cdot d)$, where n_r is the number of drifted samples. The algorithm then computes similarity, with a complexity of $\mathcal{O}(n_h \cdot d)$, and selects the most instructive samples using fast sorting, which has a time complexity of $\mathcal{O}(n_h \log n_h)$. Finally, Constrained Classifier Tuning involves fine-tuning the model with a linear classifier. The time complexity for this step is $\mathcal{O}(n_h \cdot d)$.

Combining all these stages, the overall time complexity of the entire ReCDA algorithm is: $\mathcal{O}(n_h^2 \cdot d + n_r \cdot d + n_h \log n_h)$. It is important to note that Instructive Sampling is an offline operation, and ReCDA can perform online updates at a reduced complexity with a mini-batch training strategy. Therefore, the complexity for drift adaptation will primarily depend on the size of the mini-batches used during training and feature dimension, resulting in a reduced complexity of $\mathcal{O}(n_b \cdot d)$, where n_b is the mini-batch size.

We measured the practical time and storage costs of the algorithm under the C#1 setting and compared them with other methods, as shown in Fig. 10. Since each algorithm performs a different number of iterations for drift adaptation, the time cost is measured as the time required per epoch. From the results, it is evident that ReCDA has a moderate time cost, similar to CADE, while its storage cost is the lowest. In contrast, TRIDENT incurs significant time and storage costs due to the maintenance of multiple one-class classifiers and the use of complex likelihood estimation to update the thresholds.

D. Visualization

We employ the t-SNE technique [67] to visualize original space and latent space. The visualization results on UNSW-NB15 and CICIDS-2017 datasets are presented in Fig. 9. It is observed that both benign and attack flows in recent arrivals deviate from the original data distribution, making the decision boundary of a well-trained model on the historical dataset that is challenging to generalize to drifting data. For example, column (b) shows the baseline can fit H-Benign and H-Attack on the



(a) Time Overhead (b) Storage Overhead

Fig. 10. The practical time and storage overhead at C#1.

CICIDS-2017 datasets perfectly but struggles with R-Attack. Recall that our method maps historical data and drift data from the separated original distribution to a unified latent distribution through drift-aware perturbation and representation alignment. Columns (c) and (d) demonstrate that H-Attack and R-Attack instances are notably closer in the enhanced representation of ReCDA compared to columns (a) and (b), indicating the drift-aware of our approach. Furthermore, our proposed instructive sampling strategy selects those representative instances near the fuzzy boundary to provide the classifier with discriminative knowledge, facilitating the construction of a drift-robust decision boundary.

VI. Limitation and Future Work

While ReCDA demonstrates promising adaptability and robustness to concept drift in network intrusion detection, it is important to acknowledge several limitations of its current design and inspire future research.

The current implementation of ReCDA adopts a binary classification setting, distinguishing between benign and malicious traffic. As a result, ReCDA cannot provide fine-grained identification of specific attack types. This stands in contrast to certain multiclass or hierarchical methods, such as TRIDENT, which are capable of further classifying the exact category of malicious activity. However, as noted in prior work, multi-class systems often incur significantly higher detection latency and computational overhead due to the need to maintain and

update multiple class-specific models—especially in dynamic environments where new attack variants frequently emerge. The binary setting may limit the system's utility in scenarios that require detailed attack attribution and tailored response strategies. For example, distinguishing between different types of attack vectors can be critical for forensic analysis or for deploying automated countermeasures. Additionally, although ReCDA is designed to be label-efficient and robust to concept drift, its performance may still be affected by extreme distribution shifts or by previously unseen attack types that are completely different from the historical data.

For future work, we plan to extend ReCDA to support multiclass classification, enabling it to not only detect whether traffic is malicious but also to identify the specific attack type. We believe that introducing such fine-grained detection, while maintaining efficiency and adaptability, will further enhance the practical value of our approach.

VII. Conclusion

The paper introduces ReCDA, a novel method aimed at addressing the challenges of concept drift adaptation in network intrusion detection. ReCDA incorporates drift-aware perturbation and self-supervised representation alignment to facilitate the learning of robust representations that are both drift-aware and drift-invariant. This approach avoids labor-intensive manual labeling and mitigates the risk of label noise. Additionally, ReCDA integrates an instructive sampling strategy that is meticulously designed to enhance the classifier's discriminatory capabilities between benign and malicious activities, leveraging the robust representations extracted. To comprehensively evaluate ReCDA, we surpassed conventional settings, devising a more realistic and challenging evaluation scenario. The experimental findings underscore the superior adaptability and robustness exhibited by ReCDA, positioning it as a promising candidate for addressing concept drift in network intrusion detection.

References

[1] S. Yang, X. Zheng, J. Li, J. Xu, X. Wang, and E. C. Ngai, "ReCDA: Concept drift adaptation with representation enhancement for network intrusion detection," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 3818–3828.

[2] D. Chou and M. Jiang, "A survey on data-driven network intrusion detection," *ACM Comput. Surv.*, vol. 54, no. 9, pp. 1–36, 2021.

[3] C. Xu, J. Shen, and X. Du, "A method of few-shot network intrusion detection based on meta-learning framework," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 3540–3552, 2020.

[4] A. Aldweesh, A. Derhab, and A. Z. Emam, "Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues," *Knowl.-Based Syst.*, vol. 189, 2020, Art. no. 105124.

[5] S. Yang, X. Zheng, J. Li, J. Xu, and E. C. H. Ngai, "Multi-scale contrastive attention representation learning for encrypted traffic classification," in *Proc. 33 rd ACM Int. Conf. Inf. Knowl. Manage.*, 2024, pp. 4173–4177.

[6] R. Xie et al., "Rosetta: Enabling robust TLS encrypted traffic classification in diverse network environments with TCP-Aware traffic augmentation," in *Proc. 32nd USENIX Secur. Symp.*, Anaheim, CA, USA, 2023, pp. 625–642.

[7] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-Graphsage: A graph neural network based intrusion detection system for IoT," in *Proc. NOMS IEEE/IFIP Netw. Operations Manage. Symp.*, 2022, pp. 1–9.

[8] Z. Ding et al., "MF-NET: Multi-frequency intrusion detection network for internet traffic data," *Pattern Recognit.*, vol. 146, 2024, Art. no. 109999.

[9] M. Eskandari, Z. H. Janjua, M. Vecchio, and F. Antonelli, "Passban IDS: An intelligent anomaly-based intrusion detection system for IoT edge devices," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 6882–6897, Aug. 2020.

[10] A. Derhab, M. Belaoued, I. Mohiuddin, F. Kurniawan, and M. K. Khan, "Histogram-based intrusion detection and filtering framework for secure and safe in-vehicle networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 3, pp. 2366–2379, Mar. 2022.

[11] S.-Y. Kuo, F.-H. Tseng, and Y.-H. Chou, "Metaverse intrusion detection of wormhole attacks based on a novel statistical mechanism," *Future Gener. Comput. Syst.*, vol. 143, pp. 179–190, 2023.

[12] S. Yang, X. Zheng, Z. Xu, and X. Wang, "A lightweight approach for network intrusion detection based on self-knowledge distillation," in *Proc. IEEE Int. Conf. Commun.*, 2023, pp. 3000–3005.

[13] N. Kaloudi and J. Li, "The AI-based cyber threat landscape: A survey," *ACM Comput. Surv.*, vol. 53, no. 1, pp. 1–34, 2020.

[14] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 12, pp. 2346–2363, Dec. 2019.

[15] F. Bayram, B. S. Ahmed, and A. Kassler, "From concept drift to model degradation: An overview on performance-aware drift detectors," *Knowl.-Based Syst.*, vol. 245, 2022, Art. no. 108632.

[16] O. A. Wahab, "Intrusion detection in the IoT under data and concept drifts: Online deep learning approach," *IEEE Internet Things J.*, vol. 9, no. 20, pp. 19706–19716, Oct. 2022.

[17] L. Yang et al., "{CADE}: Detecting and explaining concept drift samples for security applications," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2327–2344.

[18] F. Barbero, F. Pendlebury, F. Pierazzi, and L. Cavallaro, "Transcending transcend: Revisiting malware classification in the presence of concept drift," in *Proc. 2022 IEEE Symp. Secur. Privacy*, 2022, pp. 805–823.

[19] X. Zheng, S. Yang, E. C. Ngai, S. Jana, and L. Cavallaro, "Learning temporal invariance in android malware detectors," 2025, *arXiv:2502.05098*.

[20] X. Shu, D. Yao, and E. Bertino, "Privacy-preserving detection of sensitive data exposure," *IEEE Trans. Inf. Forensics Secur.*, vol. 10, no. 5, pp. 1092–1103, May 2015.

[21] L. L. Minku and X. Yao, "DDD: A new ensemble approach for dealing with concept drift," *IEEE Trans. Knowl. Data Eng.*, vol. 24, no. 4, pp. 619–633, Apr. 2011.

[22] L. Yang, D. M. Manias, and A. Shami, "PWPAE: An ensemble framework for concept drift adaptation in IoT data streams," in *Proc. 2021 IEEE Glob. Commun. Conf.*, 2021, pp. 01–06.

[23] X. Wang, "Enidrift: A fast and adaptive ensemble system for network intrusion detection under real-world drift," in *Proc. 38th Annu. Comput. Secur. Appl. Conf.*, 2022, pp. 785–798.

[24] Y. Chen, Z. Ding, and D. Wagner, "Continuous learning for android malware detection," in *Proc. 32nd USENIX Conf. Secur. Symp.*, 2023, pp. 1127–1144.

[25] S. Yoon, Y. Lee, J.-G. Lee, and B. S. Lee, "Adaptive model pooling for online deep anomaly detection from a complex evolving data stream," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 2347–2357.

[26] G. Andresini, F. Pendlebury, F. Pierazzi, C. Loglisci, A. Appice, and L. Cavallaro, "Insomnia: Towards concept-drift robustness in network intrusion detection," in *Proc. 14th ACM workshop Artif. Intell. Secur.*, 2021, pp. 111–122.

[27] Y. Yang, D.-W. Zhou, D.-C. Zhan, H. Xiong, and Y. Jiang, "Adaptive deep models for incremental learning: Considering capacity scalability and sustainability," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 74–82.

[28] Z. Kan, F. Pendlebury, F. Pierazzi, and L. Cavallaro, "Investigating labelless drift adaptation for malware detection," in *Proc. 14th ACM Workshop Artif. Intell. Secur.*, 2021, pp. 123–134.

[29] K. Xu, Y. Li, R. Deng, K. Chen, and J. Xu, "DroidEvolver: Self-evolving android malware detection system," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2019, pp. 47–62.

[30] N. Karim, N. C. Mithun, A. Rajvanshi, H.-P. Chiu, S. Samarasekera, and N. Rahnavard, "C-SFDA: A curriculum learning aided self-training framework for efficient source free domain adaptation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 24120–24131.

[31] X. Zheng, S. Yang, and X. Wang, "SF-IDS: An imbalanced semi-supervised learning framework for fine-grained intrusion detection," in *Proc. IEEE Int. Conf. Commun.*, 2023, pp. 2988–2993.

[32] C. Qiu et al., "3D-IDS: Doubly disentangled dynamic intrusion detection," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2023, pp. 1965–1977.
[33] H. Ding, Y. Sun, N. Huang, Z. Shen, and X. Cui, "TMG-GAN: Generative adversarial networks-based imbalanced learning for network intrusion detection," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 1156–1167, 2024.
[34] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.
[35] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proc. 2021 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 3431–3446.
[36] K. Fauvel, F. Chen, and D. Rossi, "A lightweight, efficient and explainable-by-design convolutional neural network for internet traffic classification," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2023, pp. 4013–4023.
[37] Y. Yue, X. Chen, Z. Han, X. Zeng, and Y. Zhu, "Contrastive learning enhanced intrusion detection," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4232–4247, Dec. 2022.
[38] A. F. Diallo and P. Patras, "Adaptive clustering-based malicious traffic classification at the network edge," in *Proc. IEEE INFOCOM 2021-IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
[39] N. Wang et al., "MANDA: On adversarial example detection for network intrusion detection system," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 2, pp. 1139–1153, Mar./Apr. 2023.
[40] Z. Abou El Houda, H. Moudoud, B. Brik, and L. Khoukhi, "Securing federated learning through blockchain and explainable ai for robust intrusion detection in iot networks," in *Proc. 2023 IEEE Conf. Comput. Commun. Workshops*, 2023, pp. 1–6.
[41] D. J. Kalita, V. P. Singh, and V. Kumar, "A novel adaptive optimization framework for svm hyper-parameters tuning in non-stationary environment: A case study on intrusion detection system," *Expert Syst. Appl.*, vol. 213, 2023, Art. no. 119189.
[42] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "{TESSERACT}: Eliminating experimental bias in malware classification across space and time," in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 729–746.
[43] A. Kuppa and N.-A. Le-Khac, "Learn to adapt: Robust drift detection in security domain," *Comput. Elect. Eng.*, vol. 102, 2022, Art. no. 108239.
[44] M. Dib, S. Torabi, E. Bou-Harb, N. Bouguila, and C. Assi, "EvolIoT: A self-supervised contrastive learning framework for detecting and characterizing evolving iot malware variants," in *Proc ACM Asia Conf. Comput. Commun. Secur.*, 2022, pp. 452–466.
[45] J. He, R. Mao, Z. Shao, and F. Zhu, "Incremental learning in online scenario," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 13926–13935.
[46] Y. Wu, E. Dobriban, and S. Davidson, "Deltagrad: Rapid retraining of machine learning models," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 10355–10366.
[47] A. Liu, J. Lu, and G. Zhang, "Diverse instance-weighting ensemble based on region drift disagreement for concept drift adaptation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 293–307, Jan. 2021.
[48] B. Halder, K. A. Hasan, T. Amagasa, and M. M. Ahmed, "Autonomic active learning strategy using cluster-based ensemble classifier for concept drifts in imbalanced data stream," *Expert Syst. Appl.*, vol. 231, 2023, Art. no. 120578.
[49] D. Han et al., "Anomaly detection in the open world: Normality shift detection, explanation, and adaptation," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–18.
[50] Z. Zhao, Z. Li, Z. Song, W. Li, and F. Zhang, "Trident: A universal framework for fine-grained and class-incremental unknown traffic detection," in *Proc. ACM Web Conf.*, 2024, pp. 1608–1619.
[51] B. K. Isaac-Medina, Y. F. A. Gaus, N. Bhowmik, and T. P. Breckon, "Towards open-world object-based anomaly detection via self-supervised outlier synthesis," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 196–214.
[52] Z. Zhang and M. Hoai, "Object detection with self-supervised scene adaptation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 21589–21599.
[53] K. Xu, L. Chen, and S. Wang, "Coral: Concept drift representation learning for co-evolving time-series," 2025, *arXiv:2501.01480*.
[54] A. Bardes, J. Ponce, and Y. LeCun, "Vicreg: Variance-invariance-covariance regularization for self-supervised learning," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–12.
[55] N. Wang, Y. Chen, Y. Hu, W. Lou, and Y. T. Hou, "Manda: On adversarial example detection for network intrusion detection system," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
[56] D. Han et al., "Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 8, pp. 2632–2647, Aug. 2021.
[57] J. Gu and S. Lu, "An effective intrusion detection approach using SVM with naïve bayes feature embedding," *Comput. Secur.*, vol. 103, 2021, Art. no. 102158.
[58] H. Yan et al., "Automatic evasion of machine learning-based network intrusion detection systems," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 1, pp. 153–167, Jan./Feb. 2024.
[59] A. v. d. Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," 2018, *arXiv: 1807.03748*.
[60] A. Haque, L. Khan, M. Baron, B. Thuraisingham, and C. Aggarwal, "Efficient handling of concept drift and concept evolution over stream data," in *Proc. 2016 IEEE 32nd Int. Conf. Data Eng.*, 2016, pp. 481–492.
[61] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. 2015 Mil. Commun. Inf. Syst. Conf.*, 2015, pp. 1–6.
[62] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The cicids2017 case study," in *Proc. 2021 IEEE Secur. Privacy Workshops*, 2021, pp. 7–12.
[63] D. Alvarez-Melis and N. Fusi, "Geometric dataset distances via optimal transport," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 21428–21439.
[64] M. Dragoi, E. Burceanu, E. Haller, A. Manolache, and F. Brad, "Anoshift: A distribution shift benchmark for unsupervised anomaly detection," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 32854–32867.
[65] C. A. De Souza, C. B. Westphall, R. B. Machado, J. B. M. Sobral, and G. D. S. Vieira, "Hybrid approach to intrusion detection in fog-based IoT environments," *Comput. Netw.*, vol. 180, 2020, Art. no. 107417.
[66] M. H. L. Louk and B. A. Tama, "Dual-IDS: A bagging-based gradient boosting decision tree model for network anomaly intrusion detection system," *Expert Syst. Appl.*, vol. 213, 2023, Art. no. 119030.
[67] L. Van der Maaten and G. Hinton, "Visualizing data using T-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 11, pp. 2579–2605, 2008.

Shuo Yang received the BEng degree from Sichuan University, in 2020, and the MEng degree from Tsinghua University, in 2023. He is currently working toward the PhD degree with the Department of Electrical and Electronic Engineering, The University of Hong Kong. His research interests include cybersecurity, machine learning, and trustworthy artificial intelligence.

Xinran Zheng received the BS degree in electronic information science and technology from Sichuan University and the master's degree in electronic information from Tsinghua University. Her research interests include machine learning for cybersecurity and robustness.

Jinze Li received the BS degree from the Dalian University of Technology, Dalian, China, in 2020, and the MS degree from the University of Chinese Academy of Sciences, Beijing, China, in 2023. He is currently working toward the PhD degree with The University of Hong Kong. His research interests include LLM efficient generation, federated learning, and multimodal machine learning.

Jinfeng Xu received the BS degree in software engineering from Beijing University of Technology, Beijing, China, in 2023 and the BS degree in computer science from University College Dublin, Dublin, Ireland, in 2023. He is currently working toward the PhD degree with the University of Hong Kong, Hong Kong SAR, China. His research interests include recommender system, multimodal learning, and graph learning.

Xinchen Zhang received the BE degree in automation from the Harbin Institute of Technology. She is currently working toward the PhD degree with the Department of Electrical and Electronic Engineering, The University of Hong Kong. Her research interests include Internet of Things security, particularly in the area of network intrusion detection.

Edith C. H. Ngai (Senior Member, IEEE) Edith C. H. Ngai is currently an Associate Professor in the Department of Electrical and Electronic Engineering, The University of Hong Kong. Before joining HKU in 2020, she was an Associate Professor in the Department of Information Technology, Uppsala University, Sweden. Her research interests include Internet of Things, edge intelligence, and smart cities. She was a VINNMER Fellow (2009) awarded by Swedish Governmental Research Funding Agency VINNOVA. Her co-authored papers received a Best Paper Award in QShine 2023, Best Paper Runner-Up Awards in ACM BuidSys 2024, ACM/IEEE IPSN 2013, and ACM/IEEE IWQoS 2010. She was an Area Editor of IEEE Internet of Things Journal from 2020 to 2022. She is currently an Associate Editor in *IEEE Transactions of Mobile Computing*, *IEEE Transactions of Industrial Informatics, IEEE Network, Ad Hoc Networks, and Computer Networks*. She has served as a program chair in IEEE GreenCom 2022, IEEE/ACM IWQoS 2024, and IEEE CloudCom 2025. She received a Meta Policy Research Award in Asia Pacific in 2022. She was selected as one of the N^2 Women Stars in Computer Networking and Communications in 2022. She was a distinguished Lecturer in IEEE Communication Society in 2023-2024. She is a Senior Member of ACM.